Top 10 Core Concepts of .NET Core

Pankaj Ikhar

Solution Architect | Mentor |Coach

Follow for Career Growth & Interview Preparation

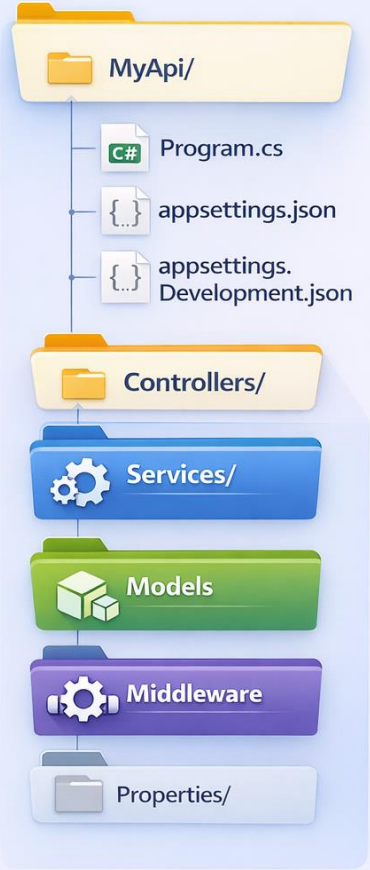
Greenleaf Educations

https://edu.greenleafhubenterprises.com/

Application Structure
& Hosting Model in NET Core

1. Basic Folder Structure

Typical Web API Project Structure:



1 Program.cs

The entry point of the application.

This is where:

- Services are registered
- Middleware is configured
- App is started

2 appsettings.json

Stores configuration like:

- Database connection strings
- API keys
- External service URLs

3 Controllers Folder

Contains API endpoints.

4 Services Folder

Contains business logic.

5 Services Folder

Contains business logic.

5 Middleware Folder

Contains custom middleware classes.

Modern Hosting Model .NET 6+

Everything is now inside **Program.cs** (Minimal API Style)

```
var builder = WebApplication.CreateBuilder(args);
// 1. Register services
builder.Services.AddControllers();
```

Application Start

Create Builder

```
var builder = WebApplication.CreateBuilder(args);
```

- Creates Configuration system
- Sets up Logging
- Prepares Dependency Injection (DI) container
- Prepares Kestrel web server internally

Kestrel is the internal web server used by ASP.NET Core.

2 Step 2

Register Services

```
builder.Services.AddControllers();
```

- Registers Controllers API
- Registers custom services
- Builds the DI container

3 Step 3

Build Application

```
var app = builder.Build();
```

- Finalizes service container
- Creates middleware pipeline
- Prepares application for requests

4 Step 4

Configure Middleware

Sets up HTTP request pipeline

- Order matters here.

5 Step 5

App Application

Starts Kestrel web server

- Listens for HTTP requests
- Blocks main thread

Application Lifecycle

Application Starts
↓
Create Builder
↓
Register Services (DI)
↓
Build Application
↓
Configure Middleware
↓
Start Web Server (Kestre!)
↓
Wait For Requests

Kestrel web server runs internally.

Services & Dependency Injection (DI)

Service Lifetimes

Lifetime	Behavior
Transient	New instance every time requested
Scoped	One instance per HTTP request
Singleton	One instance for entire app lifetime

Warning: Injecting Scoped into **Singleton** causes runtime error,
Scoped object lives per request **while** **Singleton** object lives entire app lifetime

Keyed Dependency Injection

Allows registering multiple implementatos of same interface with keys (introduced in .NET 7).

Register Keyed Services

```
builder.Services.AddKeyedScoped<
    IPaymentService("razorpay");
```

```
builder.Services.AddKeyedScoped<
    IPaymentService, StripeService);
```

- Loosely coupled
- Easily testable
- Replaceable implementation

Inject Keyed Service

```
public class PaymentController
    : ControllerBase
```

```
{ private reedonly IPaymentService
  _paymentService

  Public class PaymentController(
   IFromKeyedServices("stripe")
     paymentService payment
}
```

Architect-Level Design

Multi-tenant systems: for a rel-world example:

```
builder.Services.AddKeyedScoped<IDbConnection, SqlConnection("tenant1");
builder.Services.AddKeyedScoped<IDbConnection, MySqlConnection("tenant2");
```

Select correct DB based on tenant key.

Configuration & AppSettings Management

1. What is Configuration?

Externalized settings that control application behavior without changing code.

Examples:

- Database connection strings
- API keys
- External service URLs
- Feature flags
- Logging levels

Default Providers:

- appsettings.json
- appsettings.{Environment}.json
- User Secrets (Development)
- Environment Variables
- Command-line arguments

Architect-level clarity.

Important Concept Later providers **override** earlier ones.

Example: appsettings.json → overidem.lsy → appsettings.Production.json → Environment Variables →

3. appsettings.json Structure

Using **IConfiguration**

Accessing Configuration

```
var connectionString = builder.Configuration["ConnectionStrings:DefaultConnection];
```

Colon (:) represents hierarchy.

5. Options Pattern (Recommended Approach)

Strongly typed configuration binding.

Step 1 – Create POCO Class

```
public class EmailSettings
  public string Host { get set; }:
  public int Port { get set; }:
```

Step 2 – Bind in Program.cs

```
builder.Services.ConfiguresEmailSettings(
builder.Configuration.GetSection("
```

Inject in Service

```
public class EmailService
{ private reedornly EmailSettings
  __settings
{ public class EmailSer-
  public services.AddKeyedService()
    __Settings = options.Value,';
}
```

IOptions vs IOptionsSnapshot vs IOptionsMonitor

Interview Gold for .NET Developers

Type	Scope	When to Use
IOptions	Singleton	Static config
IOptions Monitor	Scoped	Per-request refresh
IOptions Monitor	Singleton	Real-time updates

Real Understanding

- **IOptions** → Read once at startup
- **IOptions**Snapshot → Updated per request
- **IOptions**Monitor → Updated instantly when file changes

7. Environment-Based Configuration

ASP.NET Core uses environment variable:

```
ASPNETCORE_ENVIRONMENT=Development
```

Files loaded:

- appsettings.json
- appsettings.Development.json

Environment-specific values override base config.

8. Secure Configuration (Production Ready)

9. How Configuration Loads Internally

When you run:

```
var builder = WebApplication
    CreateBuilder(args);
```

Internally:

- ConfigurationBuilder is created
- JSON providers are added
- Environment variables are loaded
- Logging configuration is bound

CreateBuilder()

Load appsettings.json → Load appsettings.json → Load environment-specific JSON

Build IConfiguration → **Build ICofiguration** → Bind to strongly typed classes

Configuration Flow Diagram

CreateBuilder() → Load appsettings.json → Load environment-specific JSON

Build IConfiguration → Bind to strongly typed classes

Pankaj Ikhar – Solution Architect | Mentor | Coach
Follow for Career Growth & Interview Preparation

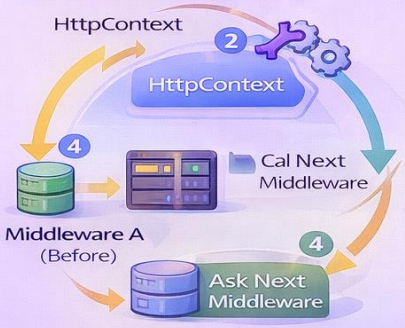
Middleware Fundamentals

Middleware is components that handle HTTP requests and modify responses before they reach the **controller** or **client**.

1. Incoming Request

Middleware is components that handle HTTP requests and modify responses before they reach the **controller** or **client**.

2. Middleware Executes



Important: Always call **await next()** unless you really want to short-circuit.

- Receives HttpContext
- Order matters
- Calls next(), or stops pipeline
- Can modify response
- Forms onion model
- Many built-in

3. How Middleware Works

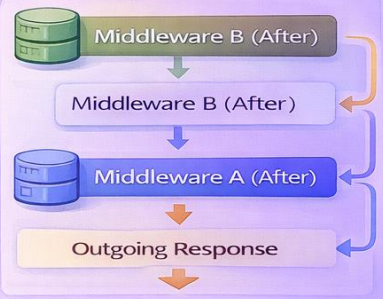
Each middleware executes:

```
app.Use(async (context, next) →
  if (!context.Request.Headers
      ContainsKey("X-API-KEY")
  {
  context.Response.StatusCode = 401;
  await context.Response.WriteAsync
    "Uanthorized")
  return;
  → await next(),
```

If Stops Pipeline

NO further **middleware** run here.

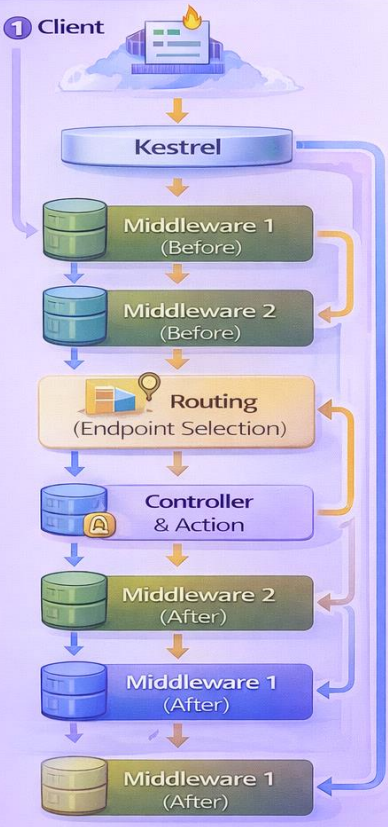
5. Outgoing Response



Important:
Always call ***await next()***
unless you really want to short-circuit.

Pankaj Ikhar – Solution Architect | Mentor | Coach
Follow for Career Growth & Interview Preparation

Full HTTP Request Pipeline in ASP.NET Core



Complete Request Lifecycle

- Receive Request
- Host Filtering
- Middleware Sequence
- Endpoint Routing
- Controller & Action
- Response Writing

Complete Request Lifecycle

- Receive Request
- Host Filtering
- Middleware Sequence
- Endpoint Routing

What This Topic:

- Order matters
- Forms "onion" model
- Response travels back

Important: Concepts

- Order matters
- Forms "onion" model
- Response travels back

Pankaj Ikhar – Solution Architect | Mentor | Coach
Follow for Career Growth & Interview Preparation

Middleware Control Methods — Use(), Run(), Map()

Understanding Use(), **Run()**, and **Map()** in ASP.NET Core

Use()

Attaches middleware to execute and call **next()**.

```
app.Use(async (context, next) →
{
   await next();
   await context.Response
          WriteAsync("END of pipeline");
};
```

Use() → next =>

- Most flexible
- Calls **next()** –continues pipeline
- Example: Logging, Auth

Flexible wrapper

Run()

Attaches **terminal** middleware and ends pipeline.

```
app.Run(async context →
{
   await context.Response
      WriteAsync("END of pipeline");
};
```

Run()

- Last **middleware**
- No next() call — ends pipeline
- Example: Custom response

Terminal endpoint

Use()

Attaches **trmminal** middleware and ends next().

```
app.Use(async context →
{
   await next();
   await context.Response.
      WriteAsync("END of pipeline")
};
```

Run()

Flexbe priplen

Run()

Routes request to a **new** middleware pipeline based on path

```
app.Map("/path", builder →
{
   // New middeware pipeline
   builder-Run(async context »
}
```

/path

/path

Pipeline A

Ends pipeline

Map()

Routes request to a new middleware pipeline

```
app.Map("/path",
{     builder →
   // New midlleware
      → admin pipeline
```

Pipeline B

Conditional branch

Run = Terminal Use = Flexible Map = Conditional Branching

Pankaj Ikhar — Solution Architect | Mentor | Coach
Follow for Career Growth & Interview Preparation

Building Custom Middleware

Step-by-Step Guide to Creating Middleware in ASP.NET Core

1 Create Middleware Class

Start by creating a middleware class in **Middleware.**

```
public class RequestLoggingMiddleware : IMiddleware
{
  public async Task InvokeAsync(HttpContext context, RequestDelegate next)
  {
     Console.WriteLine( '"Request: {context.Request.Method} { (Context.Request.Path"');
              await next(context);
     Console.WriteLine("Middleware executed.");
};
```

2 Register Middleware

Link your middleware class in **Program.cs:**

```
builder.Services.AddTransient —>
      RequestLoggingMiddleware();
```

Flexible wrapper

3 Use IApplicationBuilder

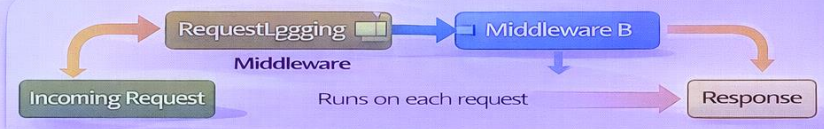
Add it to the **pipeline** using **UseMiddleware:**

```
app.UseMiddleware<RequestLogging

      app.UseRouting();
app.UseEndpoints(endpoints —>{

};     app.Run();
```

4 Your Custom Middleware is Registered!



5 Custom Middleware Code

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddTransient<RequestLoggingMiddleware);

var app = builder.Build();

app.UseMiddleware<RequestLoggingMiddleware");
      app.UseRouting();
      app.UseEndpoints(endpoints —> {
            app.Run();
};
```

Pankaj Ikhar — Solution Architect | Mentor | Coach
Follow for Career Growth & Interview Preparation

Concurrency & Asynchronous Programming in .NET APIs

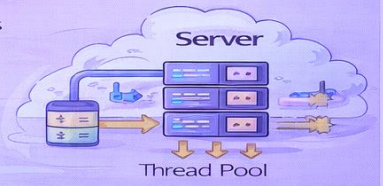
How to write efficient **async** code for ASP.NET Core APIs

1 What is Concurrency?

Handling multiple **requests simultaneously.**

- Thread pool efficiently allocates threads
- Threads freed during I/O wait
- Non-blocking I/O scales better

Executing async tasks without blocking threads



2 async / await

```
public async Task(IActionResult> Get()
{
   var data = await_repo.GetDataAsync();
   return Ok(data);
};
```

Code Tip: Makes method waitable

3 Thread Pool & Request Handling

> Thread freed during async I/O

4 Sync vs Async

- Blocks thread
- Frees thread
- Higher **scalability**

4 Benefits

- Using. Result or.**Wait()**
- Mixing sync and async
- Blocking I/O calls

5 Sync vs Async

$ **Sync** Waits at Block

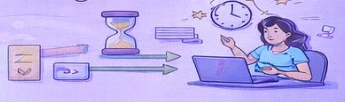
- Frees thread
- Higher scalability

Syncc thread Block

Process *{data}*
> Can run other tasks

5 Common Mistakes

- Using .**Result** or.**Wait()**
- Mixing sync and async
- Blocking I/O calls



var data = **await**. Im2ntet.MyTable ToLLSt();
Process(data); ▶ Can run other tasks

Pankaj Ikhar — Solution Architect | Mentor | Coach
Follow for Career Growth & Interview Preparation

Caching Strategies in .NET Core
(In-Memory & Distributed)

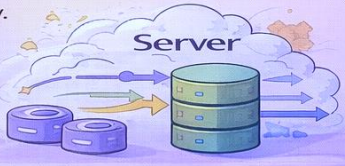
Boost performance with caching techniques in ASP.NET Core APIs

1 Why Cache Data?

Reduce database load alanc simultaneously.

- Thread pool efficiently allocates threads
- Minimize expnensive operations
- Store frequevey.c.multi—errs

Store frequently accessed data in memory for quick access



Store frequently accessed data in memory for quick access

2 In-Memory Caching

```
var dataKey ="DataKey_rid);
 if (__memoryCache.TryGetValue(),
     out var data)
 }                 Fmeudata,sc"Load datt)
 return Ok(data);
```

asynt,-it juvt Create new thread stored ron lloocmening

3 Distributed Caching

services.AddStackExchangeRedis Cache options -->

e.g., Redis

- Works across multiple servers
- Suitable for scalability & microservices

1 In-Memory Caching

- Fast & lightweight
- Suited for single server

5 Cache Expiration

- Absolute Expiration
  _absoluteExpiration = TimeSpan .FromMinutes(5);
- Sliding Expiration
  _slidingExpiration - TimeSpan (2)

```
var rata> =await trmZnlet.MyTable.ToLSt();
```

4 Caching Strategies

- Choose the right strategy In.Memory (Distributed)
- Sets proper expiration times
- Tests cache performance

5 Common Mistakes

- Using, Result or. Wait()
- Mixing sync and async
- Blocking I/O cails

Pankaj Ikhar — Solution Architect | Mentor | Coach
Follow for Career Growth & Interview Preparation

API Design & Performance

Optimizing ASP.NET Core APIs

1 Principles of Good API Design

Reduce database load simultaneously.

- Simple & Scalable Endpoints
- Use RESTful conventions
- Proper status codes
- Handle errors gracefully

Store frequently accessed data in memory for quick access

Store frequently accessed data in memory for quick access

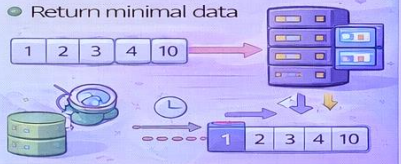
2 Optimize Data Access

- Use Async Database Calls
- Leverage EF Core"s AsNoTracking()
- Optimize queries (Select what you need)

```
public async Task(ActionResult> Get(int id)
 {
    var entity = await__dbContext.Items
        .AsNoTracking()
    FirstOrDefaultAsync(x +> .Id = id);
 },
 return Ok(entity); / No tracking, more efficient
```

3 Pagination & Filtering

- Use **Skip()/Take()**
- Support page sizes & filters
- Return minimal data



- Works across **multiple servers**
- Sufte canalysts & catory microservices

4 Cache Often

- Use In-Memory and/or Distributed Cache
- Cache costly database results and reads

Single Server

5 Avoid Common Pitfalls

- ✗ Blocking threads by sync code
- ✗ Fetching excessive data
- ✗ Abuse of "chatty" API design (too many small calls)

6 Caching Strategies

- Choose the right strategy In.Memory (Distributed)
- Profiling tools like DotTrace or BenchmarkDotNet

6 Measure & Benchmark

- ✓ Use Application Insights or Serilog
- ✓ Profiling tools like DotTrace or BenchmarkDotNet
- ✓ Optimize based on metrics and analysis

Pankaj Ikhar — Solution Architect | Mentor | Coach
Follow for Career Growth & Interview Preparation

Top 10 Core Concepts of .NET Core

Pankaj Ikhar

Solution Architect | Mentor |Coach

Follow for Career Growth & Interview Preparation

Greenleaf Educations

https://edu.greenleafhubenterprises.com/